

# GharKharch — Full Product & Technical Walkthrough

2026-09-17 · @Someone

## What GharKharch is

GharKharch (Hindi for "household expenses") is a household money app built for exactly two people who share expenses — a couple, roommates, or a small family unit — who want to track spending together without spreadsheets or splitting-app friction. It runs as an installable web app (PWA) on phone, tablet, and desktop, works fully offline for new entries, and is built specifically around Indian household spending patterns: rupee amounts, UPI/card/cash payment tracking, and merchants like D-Mart, Zudio, Swiggy, and Zomato pre-loaded as smart defaults.

The core idea: logging an expense should take under five seconds, and every number the app shows should be real and explainable — never a guessed or hallucinated figure, even when AI is involved in the experience.

**Who it's for:** couples and small households in India who currently track money informally (or not at all) and want shared visibility into where money goes, without the overhead of full accounting software or the awkwardness of a bill-splitting app built for one-off group trips.

**The problem it solves:** two people spending from a shared pool of money rarely agree on "where did it all go" at month's end. GharKharch answers that in real time — who paid, for what, how it compares to last month, and whether a category is trending over budget — automatically, from data that's already being entered.

## Getting started — step by step

### 1. Sign up

Go to `/signup`, enter an email and password. GharKharch also supports magic-link sign-in (no password needed) from the login screen for return visits.

### 2. Confirm your email

A confirmation email is sent; clicking it completes the Supabase auth handshake and lands you back in the app, signed in.

### 3. Onboarding — create or join a household

On first login you land on `/onboarding` with two choices:

- **Create a household** — you become the owner, and the app auto-seeds it with ~16 common Indian merchants (D-Mart, Zudio, Croma, Swiggy, Uber, and more) already mapped to the right categories, so Quick Add has something useful from day one.
- **Join a household** — enter the invite code your partner shares with you. A household is capped at exactly two members, matching the app's "two people sharing one pool of money" design.

### 4. Set up your profile (optional but recommended)

From **More → Profile**: add a display name, an `@username`, and a photo (crop-before-upload is built in). This is what shows up in "who paid" comparisons on the dashboard.

### 5. Log your first few expenses

Tap the floating **+ Add** button (mobile) or the **+ Add Expense** button in the sidebar (tablet/desktop). Fastest ways in:

- Type it in plain English — "Milk 60" or "Zomato 350 upi" — and let the parser fill the form.
- Or fill the form manually: item, amount, category (auto-suggested), merchant, who paid, payment method.

### 6. Set your household's payment methods

Under **More → Payment Methods**, add labels for the cards, UPI IDs, and bank accounts you'll be tagging expenses with (never actual card numbers — just names you'll recognize, like "HDFC Debit" or "Family GPay").

### 7. Set a first budget

On **More → Budgets**, set a monthly cap for a category you know runs high (groceries or eating out are common first picks). The progress bar and pacing warning only get useful once you have a few weeks of real data behind them.

### 8. Add anything recurring

Rent, subscriptions, EMIs — add them once under **More → Recurring** so the Dashboard's Daily Brief starts surfacing "due in 3 days" reminders.

## Tips for the first week

- Log expenses the moment they happen, not at day's end — the five-second Quick Add / natural-language entry is built for that; batching at night is where most tracking habits fail.
- Don't worry about perfect categorization early — the suggestion engine gets sharper the more you use it (it learns from your own patterns, not a generic model).
- Try asking **Ask GharKharch** something in week one, like "how much have we spent this week" — it's the fastest way to see the payoff before a month of data has built up.
- Install it to your home screen (your browser will prompt you, or use "Add to Home Screen") so it opens like a native app and keeps working when you lose signal mid-shop.

## Complete feature walkthrough

### Adding expenses — the core loop

- **Quick Add bar:** a horizontally-scrolling row of your household's most-used items, ranked by a real scoring formula (recency + frequency + day-of-week pattern + who usually buys it + how often at that merchant). Tap one, it's logged — no form.
- **Natural-language entry:** type "Milk 60" or "Croma 18999 card" or "Petrol 1000 yesterday" and the app parses amount, merchant, category, and payment method itself, with an AI-assisted fallback when the plain parser can't find an amount.
- **Smart amount memory:** as you type an item name, the app shows your last price paid and your typical range for that item, with a one-tap "use ₹X" chip — it never auto-fills, you stay in control.
- **Repeat / "same as last time":** one tap on any past expense clones it with today's date — for the coffee you buy every morning.
- **Shopping mode:** a rapid multi-item entry screen for a full grocery run — add item after item with a running total and count, save the whole basket at once.
- **Receipt scanning:** photograph or upload a receipt; AI reads merchant, amount, date, and line items and prefills the entry — but always through an editable review screen, never saved automatically.
- **Category & merchant smart suggestion:** a six-tier fallback (exact past pattern → exact item → exact merchant → ~50-keyword Indian-household map → household frequency → none) picks a likely category automatically, with a visible reason and confidence.

## Dashboard

One real-time view: today/week/month spend, category breakdown with animated bars, a spending trend chart, who-paid-what split between the two household members, top merchants ("Where we shop"), a spending calendar heatmap, and a **Daily Brief** card — a plain-English morning summary combining today's and this week's spend, what changed vs. last month, your next upcoming bill, and quick-add shortcuts. A separate **Forecast** card projects month-end spend from the pace set so far.

## Analytics

A tabbed deep-dive: Overview, Categories, Merchants, Items, Calendar, plus a dedicated **Spending Intelligence** page (highest-spending day/weekday, average daily/weekly spend, household-vs-personal split, fastest-growing and fastest-shrinking categories/merchants/items). Every merchant and category shows its share of total spend, trend line, and month-over-month change — not just a total.

## "Analyze this expense"

Tap any single expense to see it in context: what share of this month's category budget it represents, whether this merchant is pricier than your usual average there, whether this pushed the month's total meaningfully, similar past purchases, and a flag if the price moved unexpectedly.

## Budgets

Set a monthly cap per category or for the whole household. Progress bars show spend-to-date against the cap, with a projected month-end line (based on your pace so far) and a warning color when you're on track to exceed it — before the month is even over.

## Recurring expenses

Rent, subscriptions, EMIs — set once, and the app tracks upcoming due dates and one-tap "mark as paid" logging that automatically advances the next due date (daily/weekly/monthly/yearly/custom).

## Payment tracking

Log every expense against a specific card, UPI ID, bank account, or cash — never storing full card numbers, CVVs, PINs, or passwords, only labels ("HDFC Debit", "GPay") for tracking purposes. Payment analytics break down spend by card, by UPI app, and cash vs. digital.

## Reports & data portability

Monthly report view, CSV export, JSON export, a full household backup export (every table, schema-versioned), CSV re-import with row-by-row validation and error reporting, and a real PDF report generator (branded header, category breakdown, top purchases, month-over-month comparison, household split).

## "Ask GharKharch" — the AI assistant

A chat screen where you can ask things like "how much did we spend on groceries last month", "what's our most frequent expense", "show recurring bills", or "expenses above ₹5000 this week" in plain English, and get a real, data-backed answer — with a typing animation and sound effects for feel, and the underlying numbers always shown alongside the AI's phrasing so you can verify it yourself.

## Duplicate & data cleanup tools

Find and merge duplicate categories or merchants (including duplicate global defaults), merchant alias grouping (e.g. "Instamart" folds into "Swiggy"), bulk deactivate/delete with confirmation dialogs, and drag-and-drop category reordering.

## Household & profile

Exactly two members per household, invite-code joining, per-person profile photos (with crop-before-upload), customizable categories/merchants per household (hide or tweak global defaults without affecting other households).

## Offline & installable

Installs to your home screen like a native app. New expenses entered offline queue locally and sync automatically the moment you're back online — nothing is ever silently lost or falsely marked "saved."

## The smart intelligence layer, in plain language

GharKharch's guiding rule is **"AI explains, never calculates."** Every number shown anywhere in the app comes from a deterministic database calculation first. AI (OpenAI's `gpt-4o-mini`) is only ever layered on top to phrase things naturally or read a receipt image — and every AI-touching feature has a working fallback with no AI at all, so the app is fully functional with zero OpenAI API key configured.

What this looks like in practice:

- **Ask GharKharch** answers your question by first detecting your intent from keywords (no LLM call yet), running the actual database query, building a plain factual sentence from the real numbers, and only then optionally asking the AI to rephrase that sentence more naturally — the AI is never shown raw data and asked to “figure out” the answer itself.
- **Receipt scanning** sends a photo to the AI to extract merchant/amount/items, but the result always lands on an editable review screen — nothing saves until a human confirms it.
- **Price-change detection, spending forecasts, budget pacing, and “where did spending change”** are pure math over your history (10%+ price move on 3+ prior purchases, day-of-month spend rate, month-over-month category deltas) — no AI involved at all, so they’re exactly reproducible and explainable.
- **Anomaly and recurring-expense detection** use statistical thresholds (an expense over 2.5x your average AND above your historical highest; gap-consistency analysis across purchase dates) — again, plain math, surfaced as gentle, dismissible suggestions rather than alarms or automatic actions.

The net effect for a household: the app feels genuinely smart — it remembers what you usually pay, tells you when a price crept up, nudges you about a bill that’s due, and answers questions in your own words — while every number behind that feeling stays fully auditable and never fabricated.

## Technical architecture (developer overview)

### Stack

| Layer         | Choice                                                                                                                                                                                                                                              |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework     | Next.js 16 (App Router, Turbopack), TypeScript strict                                                                                                                                                                                               |
| Styling       | Tailwind CSS v4 (CSS-native <code>@theme</code> , no <code>tailwind.config.js</code> )                                                                                                                                                              |
| UI primitives | Radix UI + <code>vaul</code> (drawers), hand-authored shadcn-style components (no network access to <code>ui.shadcn.com</code> in the build environment, so <code>components.json</code> and <code>src/components/ui/*</code> were written by hand) |

| Layer       | Choice                                                                                                                                     |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Backend     | Supabase — Postgres + Row-Level Security + Auth + Storage, accessed via <code>@supabase/supabase-js</code> and <code>@supabase/ssr</code>  |
| Forms       | React Hook Form + Zod                                                                                                                      |
| Charts      | Recharts                                                                                                                                   |
| Motion      | Framer Motion, with a shared timing/easing token system and full <code>prefers-reduced-motion</code> support                               |
| Drag & drop | <code>@dnd-kit</code> (category reordering)                                                                                                |
| PDF export  | <code>jspdf</code> (real generated PDFs, not just browser print)                                                                           |
| AI          | Plain <code>fetch()</code> to OpenAI's Chat Completions REST endpoint ( <code>gpt-4o-mini</code> ) — no <code>openai</code> SDK dependency |
| Fonts       | Self-hosted <code>@fontsource/inter</code> (Google Fonts network access was blocked in the build environment)                              |
| Deployment  | Vercel, from a GitHub repo                                                                                                                 |

## Server action pattern

Every mutation and data fetch goes through one discriminated-union pattern:

```
type ActionResult<T> = { success: true; data: T } | { success: false; error: string };
```

implemented once in `lib/actions/auth-helpers.ts` as `runAction()` (wraps the function body, catches `ActionError`, returns a typed result) and `requireHouseholdContext()` (resolves the authenticated Supabase client, the current user id, and the household id — refusing if the user isn't a member). Every one of ~20 action files ( `expenses.ts`, `analytics.ts`, `budgets.ts`, `recurring.ts`, `receipts.ts`, `ai-assistant.ts`, etc.) is built on these two primitives, so error handling and auth-scoping are consistent everywhere rather than re-implemented per feature.

## Data flow: SSR + client cache

Every page follows the same split: a thin server `page.tsx` does one server-side data fetch and passes it as `initialData` into a "use client" `*PageClient` component, which then manages an in-memory client-side cache (`getClientCachedData / setClientCachedData / invalidateClientCache`). This means first paint is server-rendered with real data (no loading flicker), while subsequent navigation and filter changes feel instant from cache, with a clear invalidation path on every write.

## Database

19 sequentially numbered SQL migration files in `supabase/migrations/`, each idempotent, with a maintained `README.md` documenting exact run order and dependencies. All analytics are computed by ~20 Postgres functions (`get_category_breakdown`, `get_merchant_breakdown`, `get_item_analytics`, `get_card_breakdown`, etc.) — the app never pulls raw rows into JavaScript to sum them; every number is aggregated in the database. Every household-scoped table is locked down by Row-Level Security via a single `is_household_member(household_id)` `SECURITY DEFINER` helper function, which avoids the classic self-referencing-policy recursion problem. Household creation and joining go through `SECURITY DEFINER` functions with no direct client-facing insert policy on membership, so membership can only change through those controlled entry points.

## Key schema decisions

- `expenses.paid_by` (always a specific person) is kept separate from `expense_type` (personal/household/shared) — resolving an ambiguity in the original product spec.
- Soft delete via `deleted_at` on expenses; categories/merchants use a `setActive` flag rather than the same soft-delete field, since they need to disappear from pickers without affecting historical expense records.
- Categories are hierarchical (`parent_id`), with `household_id IS NULL` meaning a global default any household can use or hide.
- Amounts are `numeric(12,2)` everywhere — never floating point.
- Payment tracking (`payment_methods`, `user_cards`, `upi_profiles`, `bank_accounts`) stores labels only — never full card numbers, CVV, PIN, UPI PIN, or netbanking passwords.

## Offline architecture

Hand-written (no `next-pwa` dependency): `public/sw.js` (cache-first for static assets, network-first with an offline fallback page for navigations, and explicitly never intercepts non-GET, Server Action, or `/api/` requests), plus a native IndexedDB queue (`lib/offline/db.ts` + `offline-queue.ts`) for new expenses entered offline, which sync automatically on reconnect — deliberately scoped to *new* expenses only, since editing an existing expense offline risks a stale-data conflict.

## Code quality gates actually run

`npx tsc --noEmit` and `npx eslint .` are run after every batch of changes and kept clean. There is currently no automated test suite (unit or E2E) in the project — a deliberate, explicit deferral until the app's feature set stabilizes.

## Security, privacy & offline design

- **Row-Level Security everywhere:** every household-scoped table enforces access at the database level via Postgres RLS, not just in application code — a compromised or buggy API route still can't leak another household's data.
- **No sensitive financial credentials ever stored:** full card numbers, CVV, PIN, UPI PIN, and netbanking passwords are never captured in the schema or the UI — only user-chosen labels for identification ("HDFC Debit ending in labels the user sets", not the actual card number).
- **Private receipt storage:** the receipts Storage bucket is private (not public), with RLS policies scoped by household membership; images are served through short-lived (5-minute) signed URLs minted server-side on demand, never a persisted public link.
- **Soft delete by default:** categories and merchants are deactivated, not hard-deleted, so historical expense data is never silently orphaned or lost through a cleanup action.
- **AI never sees more than it needs, and never decides:** the AI layer only ever receives already-computed, already-scoped facts to phrase — it has no direct database access and cannot take an action on its own.
- **Offline-first for entry, not full offline-first for everything:** new expenses queue in IndexedDB and sync automatically when connectivity returns, confirmed only once the server actually accepts them; editing an existing expense while offline is deliberately routed straight to the server rather than queued, avoiding a conflict between two edited versions of the same record.

## Current status & what's next

All features described above are built and committed to the live codebase (19 database migrations, ~20 server action modules, every screen listed). `tsc --noEmit` and `eslint` both pass clean. Migrations are prepared as ready-to-run SQL files but are run manually by the project owner against the live Supabase project, by design — nothing here executes database changes automatically.

Honestly stated gaps, not glossed over:

- No automated test suite (unit or end-to-end) exists yet.
- No production build, full QA pass, or accessibility/contrast audit has been run against this exact feature set yet.
- A handful of the newest migrations (analytics depth, receipts, duplicate-merge fix) haven't been executed against the live database yet.
- Budget "pacing" language exists as a computed projection with a warning marker, but isn't labeled as "pacing" in the UI copy itself.

None of these affect what's demonstrably working today — they're the remaining steps before a first public/production release, not missing functionality.